

Projet de Programmation – ENSAE 1A

Palazuelo Jean & Razki Othmane

Année académique 2025–2026

Responsable : Simon Maura

Table des matières

1	Chemin de temps minimal	2
1.1	Présentation du problème	2
1.2	Cas sans fatigue : graphe simple	2
1.2.1	Description de l'algorithme	2
1.2.2	Analyse de complexité	2
1.3	Cas général : graphe étendu	2
1.3.1	Formulation équivalente	2
1.3.2	Implémentation	3
1.3.3	Analyse de complexité	3
1.4	Graphe implicite (GraphImplicit)	3
1.4.1	Motivation et description	3
1.4.2	Complexité	3
2	Améliorations algorithmiques	4
2.1	Élagage Pareto (<i>pruning</i>)	4
2.1.1	Idée	4
2.1.2	Implémentation	4
2.1.3	Analyse de complexité	4
2.2	Algorithme A*	4
2.2.1	Idée	4
2.2.2	Heuristique utilisée	4
2.2.3	Analyse de complexité	5
3	Extension : Point de repos optimal	5
3.1	Description du problème	5
3.2	Formalisation	5
3.3	Algorithme	5
3.4	Implémentation	6
3.5	Analyse de complexité	6
3.6	Justification de la correction	6
3.7	Illustration des résultats	6
4	Tests et résultats expérimentaux	7
4.1	Protocole de test	7
4.2	Validation sur <code>small.txt</code>	7
4.3	Performances sur fichiers de taille croissante	7
4.4	Analyse des résultats	7
	Conclusion	8

Partie 1. Chemin de temps minimal

1.1. Présentation du problème

On considère un graphe orienté $G = (V, E)$ représentant un réseau routier. Chaque arête $e = (i, j) \in E$ possède deux attributs entiers : une longueur $\ell(e) \geq 1$ et un coefficient d'augmentation de fatigue $f(e) \geq 0$. Un agent part du sommet v_s avec une fatigue initiale nulle. Le temps de parcours d'un chemin $(v_0, v_1, \dots, v_k)$ est défini par :

$$\text{temps}(v_0, \dots, v_k) = \sum_{i=1}^k \ell(v_{i-1}, v_i) \times \left(1 + \sum_{j=1}^{i-1} f(v_{j-1}, v_j) \right)$$

L'objectif est de trouver le chemin minimisant ce temps entre v_s et v_t .

1.2. Cas sans fatigue : graphe simple

1.2.1 Description de l'algorithme

Lorsque $f(e) = 0$ pour toute arête, le coût d'un chemin est simplement la somme des longueurs. On se ramène à un problème classique de plus court chemin dans un graphe pondéré, résolu par l'algorithme de Dijkstra.

Nous avons implémenté deux variantes :

- `shortest_path_naïve` : version sans file de priorité. À chaque itération, le sommet non visité de distance minimale est trouvé par un parcours linéaire de la liste `non_visite`.
- `shortest_path` : version optimisée utilisant un tas binaire (`heapq`).

La méthode `build_simple_graph` de la classe `Network` construit un objet `Graph` en ignorant les coefficients de fatigue, ne retenant que les paires (voisin, longueur).

1.2.2 Analyse de complexité

Soit $n = |V|$ le nombre de sommets et $m = |E|$ le nombre d'arêtes.

- **Version naïve** : la sélection du minimum dans `non_visite` coûte $O(n)$ par itération et la suppression de la liste coûte $O(n)$. La boucle principale s'exécute n fois. La complexité globale est donc $O(n^2 + m)$.
- **Version optimisée (tas binaire)** : chaque insertion ou extraction dans le tas coûte $O(\log n)$. On effectue au plus m insertions. La complexité globale est $O((n + m) \log n)$.

Pour des graphes creux ($m = O(n)$), la version optimisée est nettement plus efficace.

1.3. Cas général : graphe étendu

1.3.1 Formulation équivalente

Dans le cas général, le coût d'une arête dépend de la fatigue accumulée, qui est elle-même une fonction du chemin parcouru. Cela rompt la propriété de sous-structure optimale nécessaire à Dijkstra sur le graphe original.

On résout ce problème en construisant un **graphe étendu** $G' = (V', E')$ dans lequel chaque état est un couple (v, F) où $v \in V$ est un sommet et $F \in \{0, \dots, F_{\max}\}$ est la fatigue accumulée. On définit :

$$F_{\max} = \sum_{e \in E} f(e)$$

comme borne supérieure sur la fatigue totale atteignable.

Une arête $((u, F), (v, F'))$ existe dans G' si $(u, v) \in E$, $F' = F + f(u, v)$, et $F' \leq F_{\max}$. Son poids est $\ell(u, v) \times (1 + F)$, reflétant le ralentissement dû à la fatigue courante.

Chercher le chemin de temps minimal de $(v_s, 0)$ à $(v_t, *)$ dans G' est alors équivalent au problème original.

1.3.2 Implémentation

La méthode `build_extended_graph` de la classe `Graph` construit explicitement G' sous forme de liste d'adjacence, puis réutilise `shortest_path` sur ce graphe étendu.

1.3.3 Analyse de complexité

- Nombre de sommets de G' : $|V'| = n \times (F_{\max} + 1)$.
- Nombre d'arêtes de G' : $|E'| = O(m \times F_{\max})$.
- Construction de G' : $O(m \times F_{\max})$.
- Dijkstra sur G' : $O((|V'| + |E'|) \log |V'|) = O(m \cdot F_{\max} \cdot \log(n \cdot F_{\max}))$.

1.4. Graphe implicite (GraphImplicit)

1.4.1 Motivation et description

Construire G' explicitement peut s'avérer prohibitif en mémoire lorsque $F_{\max}$ est grand. La classe `GraphImplicit`, sous-classe de `Graph`, évite ce problème en calculant les voisins d'un état *à la volée* via une fonction `neighbours_fn`, sans jamais stocker la liste d'adjacence complète.

La méthode `build_graph_implicit` de la classe `Network` encapsule cette logique : elle définit `neighbours_fn` comme une fermeture capturant le réseau et $F_{\max}$, puis retourne un objet `GraphImplicit`. La méthode `shortest_path` héritée de `Graph` est réutilisée sans modification.

1.4.2 Complexité

La complexité en temps est identique à celle du graphe étendu explicite. En revanche, la complexité en **espace** passe de $O(n \cdot F_{\max} + m \cdot F_{\max})$ à $O(n \cdot F_{\max})$ (uniquement les états visités par Dijkstra sont conservés en mémoire).

Partie 2. Améliorations algorithmiques

2.1. Élagage Pareto (*pruning*)

2.1.1 Idée

Dans le graphe étendu, deux états (v, F_1) et (v, F_2) avec $F_1 \leq F_2$ et $t_1 \leq t_2$ (temps d'arrivée) exhibent une dominance Pareto : (v, F_2, t_2) ne peut jamais conduire à une solution meilleure que (v, F_1, t_1) , car toute continuation sera plus lente (fatigue plus élevée) et part d'un temps déjà supérieur. On peut donc **éliminer** les états Pareto-dominés lors de l'exploration.

2.1.2 Implémentation

La fonction `shortest_path_with_pruning` (définie hors classe, attachée dynamiquement via `types.MethodType`) maintient un dictionnaire `best_for_node` associant à chaque sommet le *front de Pareto* des couples (fatigue, temps) non dominés. Avant de traiter un état extrait du tas, on vérifie s'il est dominé par un état déjà connu pour ce sommet. Si oui, il est ignoré (*pruned*).

2.1.3 Analyse de complexité

Dans le pire cas (aucun élagage), la complexité reste $O(m \cdot F_{\max} \cdot \log(n \cdot F_{\max}))$, identique à Dijkstra sur G' . En pratique, le pruning réduit significativement le nombre d'états explorés, particulièrement sur des graphes denses ou avec des fatigues élevées, car de nombreuses branches sont coupées précocement.

La taille du front de Pareto pour un sommet est bornée par $\min(F_{\max} + 1, n)$ dans le pire cas, mais reste généralement petite en pratique.

2.2. Algorithme A*

2.2.1 Idée

L'algorithme A* généralise Dijkstra en guidant la recherche vers le but grâce à une **heuristique admissible** h . À chaque étape, l'état minimisant $f = g + h$ est traité, où g est le coût réel depuis l'état initial et h est une estimation du coût restant jusqu'à v_t . L'admissibilité garantit $h(\text{état}) \leq \text{coût réel restant}$, ce qui assure l'optimalité.

2.2.2 Heuristique utilisée

On choisit comme heuristique la distance minimale sans fatigue de v à v_t , calculée par un Dijkstra inversé sur le graphe simple (sans fatigue) depuis v_t lors d'un pré-traitement. Formellement, pour un état (v, F) :

$$h(v, F) = d_{\text{simple}}(v, v_t)$$

où d_{simple} est la distance dans le graphe ignorant la fatigue. Cette heuristique est bien admissible car négliger la fatigue ne peut que sous-estimer le coût réel (les coûts avec fatigue sont toujours supérieurs ou égaux à ceux sans fatigue).

2.2.3 Analyse de complexité

Le pré-traitement (Dijkstra inversé) coûte $O((n + m) \log n)$. Le A^* lui-même a une complexité qui dépend de la qualité de l'heuristique. Dans le pire cas (heuristique nulle), on retrouve la complexité de Dijkstra sur G' . En pratique, A^* explore beaucoup moins d'états que Dijkstra pur lorsque l'heuristique est informative, c'est-à-dire proche du coût réel.

Partie 3. Extension : Point de repos optimal

3.1. Description du problème

On considère un seul agent devant aller de v_s à v_t . On dispose d'un unique **lieu de repos** que l'on peut placer en n'importe quel sommet $r \in V$. Lorsque l'agent passe par r , sa fatigue est remise à zéro. L'objectif est de choisir r de façon à **minimiser le temps total** du trajet $v_s \rightarrow r \rightarrow v_t$.

3.2. Formalisation

Le temps total lorsque le lieu de repos est placé en r est :

$$T(r) = d_{\text{forward}}(r) + d_{\text{backward}}(r)$$

où :

- $d_{\text{forward}}(r)$ est le temps minimal pour aller de l'état $(v_s, 0)$ à $(r, *)$ dans le graphe étendu (fatigue quelconque à l'arrivée en r) ;
- $d_{\text{backward}}(r)$ est le temps minimal pour aller de l'état $(r, 0)$ à $(v_t, *)$ dans le graphe étendu (l'agent repart de r avec une fatigue nulle).

Le sommet optimal est $r^* = \arg \min_{r \in V} T(r)$.

3.3. Algorithme

L'algorithme repose sur **deux passes de Dijkstra** :

1. **Passe aller** : on exécute Dijkstra depuis $(v_s, 0)$ sur le graphe implicite étendu. Pour chaque sommet r , on retient le temps minimal parmi tous les états (r, F) atteints :

$$d_{\text{forward}}(r) = \min_F \text{dist_forward}[(r, F)]$$

2. **Passe retour** : on exécute Dijkstra depuis $(v_t, 0)$ sur le **graphe implicite étendu inversé**. Ce graphe renversé est construit par la méthode `build_reversed_implicit_graph` : une arête $(u, F_u) \rightarrow (v, F_v)$ du graphe original devient $(v, F_v) \rightarrow (u, F_u)$ dans le graphe inversé, avec le même coût $\ell(u, v) \times (1 + F_u)$.

Pour chaque sommet r , on récupère la distance depuis $(v_t, 0)$ dans le graphe inversé jusqu'à l'état $(r, 0)$, ce qui correspond exactement au temps minimal de $(r, 0)$ à $(v_t, *)$ dans le graphe original :

$$d_{\text{backward}}(r) = \text{dist_backward}[(r, 0)]$$

3. **Sélection** : on calcule $T(r) = d_{\text{forward}}(r) + d_{\text{backward}}(r)$ pour tout $r \in V$, et on retourne le minimum.

3.4. Implémentation

La méthode `find_optimal_rest_point` de la classe `Network` implémente cet algorithme. Elle appelle successivement `build_graph_implicit` pour la passe aller, puis `build_reversed_implicit` pour la passe retour. Le résultat est trié par $T(r)$ croissant afin de fournir un classement complet des candidats.

3.5. Analyse de complexité

Notons $n = |V|$, $m = |E|$, et $F_{\max} = \sum_{e \in E} f(e)$.

- **Graphe implicite étendu** : $|V'| = O(n \cdot F_{\max})$, $|E'| = O(m \cdot F_{\max})$.
- **Passe aller** (Dijkstra sur G') : $O(m \cdot F_{\max} \cdot \log(n \cdot F_{\max}))$.
- **Construction du graphe inversé** : lors de chaque appel à `neighbours_fn_reversed`, on parcourt toutes les arêtes du réseau pour trouver les prédécesseurs d'un nœud. Le coût par appel est $O(m)$, mais dans Dijkstra les voisins d'un état sont demandés au plus une fois, donc le coût total est $O(m \cdot |V'|) = O(m^2 \cdot F_{\max})$ dans le pire cas. Une implémentation plus efficace (pré-calcul de la liste de prédécesseurs) ramènerait ce coût à $O(m \cdot F_{\max} \cdot \log(n \cdot F_{\max}))$.
- **Calcul de $T(r)$ pour tout r** : $O(n)$.

La complexité globale de `find_optimal_rest_point` est donc :

$$O(m \cdot F_{\max} \cdot \log(n \cdot F_{\max}))$$

avec l'implémentation optimisée du graphe inversé, contre $O(m^2 \cdot F_{\max})$ avec l'implémentation actuelle (parcours exhaustif des prédécesseurs).

3.6. Justification de la correction

La correction de l'algorithme repose sur le fait que le lieu de repos remet la fatigue à zéro. Le chemin optimal avec un repos en r peut donc être décomposé en deux sous-chemins indépendants : $v_s \rightarrow r$ (fatigue quelconque) et $r \rightarrow v_t$ (fatigue nulle au départ). Ces deux sous-problèmes sont résolus de manière optimale et indépendante par les deux passes de Dijkstra.

3.7. Illustration des résultats

L'algorithme produit, pour chaque sommet candidat r , la valeur $T(r)$, ainsi que le temps de référence sans lieu de repos $T_{\emptyset} = d_{\text{forward}}(v_t)$. Le gain apporté par le lieu de repos est :

$$\Delta = T_{\emptyset} - T(r^*)$$

On constate que sur des réseaux avec des coefficients de fatigue élevés, le gain peut être substantiel, car repartir avec fatigue nulle depuis un sommet bien positionné réduit fortement le coût des arêtes restantes.

Partie 4. Tests et résultats expérimentaux

4.1. Protocole de test

Les algorithmes ont été testés sur trois fichiers de tailles croissantes : `small.txt` (4 sommets), `medium-smallfatigue.txt` et `large-largefatigue.txt`. Les tests sont implémentés avec le framework `pytest` dans le fichier `test/test_algorithms.py`.

4.2. Validation sur `small.txt`

Le réseau `small.txt` relie `lozere` à `saclay` via deux chemins possibles. Les résultats obtenus sont les suivants :

Algorithme	Résultat
Dijkstra simple (sans fatigue)	55
Dijkstra naïf (sans fatigue)	55
Dijkstra avec fatigue	125
Pruning Pareto	125
A*	125
Sans lieu de repos	125
Meilleur lieu de repos	<code>ensae</code>
Temps avec lieu de repos	55

Sans fatigue, le chemin optimal est `lozere` → `ensae` → `saclay` (coût $10 + 45 = 55$). Avec fatigue, le chemin optimal devient `lozere` → `guichet` → `ensae` → `saclay` (coût $20 + 15 + 90 = 125$), car le passage par `guichet` (fatigue nulle) évite d'arriver en `ensae` avec une fatigue de 2. Les quatre algorithmes avec fatigue donnent tous 125, ce qui valide leur cohérence.

Pour le point de repos, placer le lieu de repos en `ensae` permet de repartir avec fatigue nulle : il ne reste que `ensae` → `saclay` (coût $45 \times 1 = 45$) après avoir rejoint `ensae` sans fatigue (coût $10 \times 1 = 10$), soit un total de 55, identique au cas sans fatigue.

4.3. Performances sur fichiers de taille croissante

Les temps d'exécution mesurés sont les suivants :

Fichier	Dijkstra simple	Dijkstra fatigue	Pruning Pareto
<code>small.txt</code>	0.010 ms	0.013 ms	0.017 ms
<code>medium-smallfatigue.txt</code>	0.554 ms	4850.777 ms	6.907 ms

4.4. Analyse des résultats

Plusieurs observations se dégagent de ces mesures.

Explosion combinatoire du graphe étendu. Le Dijkstra avec fatigue est environ 8 750 fois plus lent que le Dijkstra simple sur `medium-smallfatigue.txt` (4850 ms contre 0.554 ms). Cela s'explique par l'explosion de l'espace d'états : au lieu de n sommets, on explore $n \times F_{\max}$ états, et $F_{\max}$ peut être très grand.

Efficacité du pruning Pareto. Le pruning Pareto réduit le temps de 4850 ms à 6.9 ms sur `medium-smallfatigue.txt`, soit un facteur de réduction d'environ **700**. En éliminant les états Pareto-dominés, on explore une infime fraction du graphe étendu. C'est l'amélioration la plus spectaculaire observée.

Coût du point de repos. L'algorithme du point de repos effectue deux passes de Dijkstra sur le graphe étendu, ce qui explique son temps d'exécution d'environ 10 176 ms sur `medium-smallfatigue.txt`, soit approximativement le double du Dijkstra avec fatigue seul.

Conclusion

Ce projet nous a permis d'implémenter et d'analyser une chaîne complète d'algorithmes de graphes, depuis Dijkstra naïf jusqu'à A* et l'extension au problème du point de repos. Les choix de conception – en particulier la hiérarchie de classes `Graph/GraphImplicit` et la séparation entre `Network` et `Graph` – permettent de réutiliser `shortest_path` sans modification dans tous les contextes, y compris sur le graphe étendu et le graphe inversé.

Les améliorations algorithmiques (pruning Pareto, A*) réduisent significativement le nombre d'états explorés en pratique, tout en conservant les garanties d'optimalité. L'extension du point de repos illustre comment des problèmes de planification plus complexes peuvent se réduire à deux passes de Dijkstra par une modélisation soigneuse du graphe étendu.